

CME 2201 PROJECT II PROJECT REPORT

Yusuf Bayrak

2022510101

TABLE OF CONTENTS

1. Introduction

2. Design and Implementation

2.1. Class Structure

2.2. Used Data Structures

3. Algorithms and Functionalities

3.1. Menu Class

3.1.1. Load Graph (Functionality 1)

3.1.2. Search for Protein (Functionality 2)

3.1.3. Check Interaction (Functionality 3)

3.1.4. Most Confident Path (Functionality 4)

3.1.5. Graph Metrics (Functionality 5)

3.1.6. Traversals (Functionality 6)

3.2. Vertex Class

3.3. DirectedGraph Class

3.3.1. Most Confident Path (Dijkstra Variation)

3.3.2. Breadth-First Traversal

3.3.3. Depth-First Traversal

3.3.4. Diameter Calculation (Max Hops via BFS)

3.3.5. Reciprocity Calculation

4. Conclusion

1. Introduction

In this assignment, the aim was to develop a Java program that represents the STRING Protein-Protein Interaction (PPI) Network using a Edge List Represented Graph Abstract Data Type (ADT). The main goal was to implement a directed graph structure, load protein interaction data, search information of a specific protein, filter interactions based on confidence scores, and implement various graph algorithms such as traversal (BFS, DFS), pathfinding, and metric calculations.

2. Design and Implementation

The Project was implemented in Java applying Object Oriented Programming principles.

2.1 Class Structure

Protein Class: Represents the data stored in graph nodes. It contains getter and setter methods for Protein ID, preferred name, size, and annotation. I

overrode hashCode and equals methods to ensure proteins are identified by their IDs not their locations in RAM in HashMaps.

Vertex Class: Implements the VertexInterface. It represents a node in the graph and contains a list of Edge objects (Edge list) to store connections to neighbors. It also manages variables like visited, cost, and predecessor for algorithms.

DirectedGraph Class: The class implementing GraphInterface. It manages all vertices using a HashMap. I chose HashMap because it provides $O(1)$ time complexity for retrieving vertices which is very efficient.

Menu Class: Handles the user interaction. It loads the data files, handles exceptions (try-catch), and executes commands taken from the user.

2.2 Used Data Structures

HashMap: Used to store the collection of vertices for efficient look-up by Protein ID.

LinkedList: Used in the Vertex class to store the list of edges (neighbors). Also used as a Queue for BFS.

Stack: Used for Depth-First Search and for reconstructing the path in the Most Confident Path algorithm.

PriorityQueue: Used in the Most Confident Path algorithm to always reach the highest-scored path first.

3. Algorithms and Functionalities

3.1 Menu Class

3.1.1 Load Graph (Functionality 1)

The initializeGraph method reads the 9606.protein.links.v12.0.txt file with a try-catch block. Before creating an edge, it checks the confidence score of the interaction. Interactions with a score lower than the threshold are ignored. The initialization time is also measured.

3.1.2 Search for Protein (Functionality 2)

The method searches for a protein using its specific Protein ID.

We use a `HashMap<String, Protein>` to store protein data loaded from the `9606.protein.info.v12.0.txt` file.

3.1.3 Check Interaction (Functionality 3)

This function checks if an edge exists between two given Protein IDs. The `hasEdge(begin, end)` method in the `DirectedGraph` class is used.

3.1.4 Most Confident Path (Functionality 4)

To find the most confident interaction between two proteins, I implemented a variation of Dijkstra's Algorithm which I will also address later.

3.1.5 Graph Metrics (Functionality 5)

We calculate the following metrics:

Vertex & Edge Count: Retrieved directly from the graph's size properties.

Average Degree: Calculated using the formula $2 * \text{numberOfEdges} / \text{numberOfVertices}$.

Reciprocity: Calculates using the method in `DirectedGraph` class which I will address later.

Diameter: We calculate the maximum distance (using BFS) for every node in the graph and find the maximum among them.

3.1.6 Traversals (Functionality 6)

I implemented two graph traversal algorithms starting from a user-specified origin protein:

Breadth-First Search (BFS): Implemented using a Queue. It explores the graph layer by layer.

Depth-First Search (DFS): Implemented using a Stack. It explores as far as possible along each branch before backtracking.

3.2 Vertex Class

It holds the vertex label, a list of connections (`edgeList`), and necessary fields for graph algorithms: `visited` (boolean for traversals), `cost` (double for path weights), and `previousVertex` (predecessor).

It uses a private inner class Edge to store the destination vertex and the path weight. The connect method first checks if the edge already exists and adds a new edge to the edgeList.

It provides NeighborIterator and WeightIterator to traverse connected vertices and their edge weights.

3.3 DirectedGraph Class

It implements the graph using a HashMap and an integer edgeCount to get the total number of connections in the network. It provides fundamental methods: addVertex to add unique proteins into the map, addEdge to establish edges between them using connect method of vertex class, and hasEdge to verify if a specific interaction exists.

This class also implements a variation of Dijkstra algorithm to find the most confident path, two standard graph traversal algorithms Breadth-First Traversal and Depth-First Traversal, an algorithm to find the diameter of the graph that uses BFS to find the maximum number of hops from a specific origin node and the algorithm that calculates the reciprocity of the network by verifying mutual connections between vertices.

3.3.1 Most Confident Path (Dijkstra Variation)

This algorithm finds the path with the highest confidence score between two proteins.

It starts by resetting all vertices and placing the origin vertex into a PriorityQueue.

Unlike standard Dijkstra algorithm which minimizes distance, our implementation uses a PriorityQueue ensures that the algorithm always reaches the path with the highest total confidence score first.

It iteratively finds the highest-scored vertex, marks it as visited, and updates the costs of its neighbors by adding the edge weight to the current path cost.

Once the destination vertex is reached the path is reconstructed backwards using the predecessor pointers held in each vertex.

3.3.2 Breadth-First Traversal

Explores the graph layer by layer.

It utilizes a Queue to store the order of visits.

The algorithm starts at the origin, marks it as visited, and enqueues it. It then dequeues a vertex, visits all its unvisited neighbors, marks them, and enqueues them. This cycle continues until the queue is empty.

3.3.3 Depth-First Traversal

Explores as far as possible along each branch before backtracking.

It utilizes a Stack to manage the traversal path.

The algorithm pushes the origin onto the stack and marks it as visited. It then peeks at the top vertex and looks for an unvisited neighbor.

If an unvisited neighbor is found, it is marked and pushed onto the stack.

If there is no unvisited neighbors, the vertex is popped from the stack (backtracking) to reach other paths.

3.3.4 Diameter Calculation (Max Hops via BFS)

I implemented a helper method that runs a BFS starting from a specific node. Instead of just visiting, it tracks the "hop count" (cost) for each node, incrementing the cost by 1 for each level of depth. It returns the maximum cost found from that origin.

To find the graph's diameter, the algorithm iterates through every vertex in the graph, calculates the maximum distance for each, and finds the global maximum by comparing.

3.3.5 Reciprocity Calculation

The algorithm iterates through all vertices and their outgoing edges.

For every edge from Vertex A to Vertex B, it checks if the reverse edge exists using the `hasEdge` method.

It counts the total number of such mutual edges and divides this count by the total number of edges in the graph to produce a ratio between 0 and 1.

4. Conclusion

In this project, I developed a Java program to analyze Protein-Protein Interaction networks using a Directed Graph with Edge List representation. Applying OOP principles, I utilized a HashMap for efficiency.

We successfully implemented BFS, DFS, and a Dijkstra-based algorithm to find the most confident interaction paths. We also calculated network metrics like Average Degree and Reciprocity. The primary challenge we encountered was the high computational cost of the Diameter calculation on large datasets.

